

คู่มือระบบควบคุมแขนกล Dobot Magician ด้วย ROS 1 (Noetic), OpenCV, pydobot2 และ PyQt5

การพัฒนาระบบแขนกลอัจฉริยะ คัดแยกสี หลบหลีกสิ่งกีดขวาง และเชื่อมบล็อกอัตโนมัติ (เริ่มต้นจากศูนย์)

ผู้พัฒนา / สถาปัตยกรรมระบบ: Thanadechสภาพแวดล้อมระบบ: Ubuntu (Google Deepmind Pair-Programming Linux 20.04 LTS / ROS 1 Assistant) Noetic Ninjemys	พื้นที่ทำงาน (Catkin Workspace): /root/วันที่ Dobot-Magician-Autonomous-Stacking-ROS1	เผยแพร่: กันยายน 2026
---	---	-----------------------

**วัตถุประสงค์ของเอกสาร**

เอกสารฉบับนี้จัดทำขึ้นเพื่อให้ผู้เริ่มต้นที่ไม่มีพื้นฐานด้านหุ่นยนต์หรือ ROS 1 มาก่อน สามารถเข้าใจหลักการทำงาน สถาปัตยกรรมการสื่อสาร โค้ดคอมพิวเตอร์วิทัศน์ และการควบคุมฮาร์ดแวร์แขนกล Dobot Magician ได้อย่างถ่องแท้ ตั้งแต่ทฤษฎีพื้นฐานของระบบ ROS Master การรับส่งข้อมูลผ่าน Topic การแปลงพิกัดภาพ จนถึงระบบอัตโนมัติ 3 เฟสจริงในโรงงานอุตสาหกรรม

สารบัญเนื้อหา (Table of Contents)

- **บทที่ 1:** สถาปัตยกรรมระบบและโครงสร้างสนามทำงาน (System Overview & Field Geometry)
- **บทที่ 2:** ปูพื้นฐาน ROS 1 Noetic จากศูนย์ (Master, Nodes, Topics, Parameters, Launch)
- **บทที่ 3:** ระบบประมวลผลภาพด้วย OpenCV และการส่งภาพใน ROS 1 (Vision Pipeline & Image Transport)
- **บทที่ 4:** การควบคุมแขนกล Dobot Magician ด้วย pydobot2 และโหนดควบคุม (Dobot Driver & Kinematics)
- **บทที่ 5:** การสร้างส่วนติดต่อผู้ใช้ด้วย PyQt5 (GUI Architecture, Signals/Slots, RosBridge)
- **บทที่ 6:** ตระการภารกิจ 3 เฟส และการหลบหลีกสิ่งกีดขวาง (3-Phase Mission & Collision Avoidance)
- **บทที่ 7:** คู่มือการใช้งานหน้าต่างโปรแกรม (UI Dashboard Walkthrough)
- **บทที่ 8:** คำสั่งเริ่มต้นใช้งานและการแก้ไขปัญหา (Build, Execution & Troubleshooting)

บทที่ 1: สถาปัตยกรรมระบบและโครงสร้างสนามทำงาน

1.1 ภาพรวมทางกายภาพของสนามทำงาน (Field Layout)

ระบบนี้ทำงานบนแผ่นกระดาษมาตรฐาน A4 (210 x 297 มม.) เป็นระนาบทำงานหลัก ประกอบด้วยจุดอ้างอิงสำคัญ 4 ส่วน:

1. **ฐานแบบกล Dobot Magician**: ติดตั้งอยู่ที่พิกัดศูนย์กลาง ($X = 105.0$ มม., $Y = 270.0$ มม.)
2. **ถาดตาราง 3x3 (Pallet Grid)**: ขนาดภายนอก 114.5 x 114.5 มม. มีระยะห่างระหว่างจุดศูนย์กลางช่อง (Pitch) 35.0 มม. และช่องเจาะขนาด 25.0 มม. โดยช่องรอบนอกทั้ง 8 ช่องใช้สำหรับวางบล็อกสีต่าง ๆ (แดง, เหลือง, เขียว, ฟ้า)
3. **เป้าหมาย Center Goal (ช่อง [1, 1])**: จุดศูนย์กลางตารางที่เซลล์ต้องนำบล็อกเป้าหมายมาวางซ้อนกันในแนวตั้ง (บล็อกไว้ไม่เกิน 4 ก้อน ที่ความสูง $Z = 30.0, 55.0, 80.0, 105.0$ มม.)
4. **ช่อง Feeder 4 ตำแหน่ง (Feeder 1..4)**: อยู่ฝั่งซ้ายของกระดาษ ($Y \sim +79.3$ มม.) มีวงแหวนสีเขียวกำกับ ใช้สำหรับพับบล็อกสี่เหลี่ยมขวางชั่วคราว

[4 Feeder Slots]
(Green Rims)
(#1)

(#2)

(#3)

(#4)

[3x3 Pallet Grid: 114.5 x 114.5 mm]

```
+-----+-----+-----+
| (0,0) | (0,1) | (0,2) |
+-----+-----+-----+
| (1,0) | (1,1) | (1,2) |
|       | GOAL  |       |
+-----+-----+-----+
| (2,0) | (2,1) | (2,2) |
+-----+-----+-----+
```

[Dobot Magician Robot Base]
(105, 270) mm

1.2 โครงสร้างซอฟต์แวร์ใน Catkin Workspace

Workspace ในระบบ ROS 1 (`/root/Dobot-Magician-Autonomous-Stacking-ROS1`) ถูกแบ่งออกเป็นแพ็คเกจย่อยตามหน้าที่อย่างเป็นระบบ:

- `dobot_v2` : โหนดประมวลผลภาพกล้อง (`detection_node.py`), การคำนวณพิกัด Homography, โดเวอร์ฮาร์ดแวร์แบบกล (`dobot_driver.py`), โหนดควบคุม (`dobot_controller_node.py`) และตัววางแผนเส้นทางหลัก 3 เฟส (`mission_executor.py`)
- `Dobot_UI` : หน้าจอแดชบอร์ดกราฟิก PyQt5 รับส่งข้อมูลและควบคุมการทำงานผ่าน ROS 1 แบบเรียลไทม์
- `dobot_project` : แพ็คเกจพื้นฐานสำหรับทดสอบระบบแบบโหนดเดียว

บทที่ 2: ปูพื้นฐาน ROS 1 Noetic จากศูนย์

2.1 ROS 1 Noetic คืออะไร?

ROS 1 (Robot Operating System 1 - รุ่น Noetic Ninjemys) เป็นระบบมิดเดิลแวร์สำหรับหุ่นยนต์ที่ทำงานบน Ubuntu Linux 20.04 LTS ช่วยจัดการการสื่อสารระหว่างโปรแกรมย่อย ๆ (เรียกว่า **Node**) ให้ทำงานประสานกันได้โดยไม่ต้องรวมเป็นโปรแกรมใหญ่ขึ้นเดียว

สถาปัตยกรรมแบบรวมศูนย์ (Centralized ROS Master): ใน ROS 1 จะมี `roscore` ทำหน้าที่เป็น Master Registry เปิดพอร์ต 11311 คอยดูแลว่าโหนดใดเป็นผู้ส่ง (Publisher) หรือผู้รับ (Subscriber) เมื่อโหนดแต่ละตัวเริ่มต้นขึ้น จะติดต่อ Master ผ่าน XML-RPC จากนั้นจึงสร้างการเชื่อมต่อตรงระหว่างโหนดเพื่อสตรีมข้อมูลผ่านโปรโตคอลความเร็วสูง TCPROS หรือ UDPROS

2.2 องค์ประกอบพื้นฐานใน ROS 1

1. โหนด (Node - `rospy.init_node`)

โหนดคือโปรเซสเดี่ยวที่มีหน้าที่เฉพาะทาง:

- `usb_cam` : จับภาพวิดีโอจากกล้อง USB ผ่านคอร์เนล V4L2 แล้วส่งเข้าสู่ระบบ ROS
- `detection_node` : ประมวลผลภาพ คัดแยกสี ค้นหาตารางกริด และคำนวณพิกัด 3 มิติของบล็อกล
- `dobot_controller` : สื่อสารกับแขนกล Dobot ผ่านพอร์ต Serial วางแผนเส้นทาง และควบคุมความปลอดภัย
- `dobot_ui` : แสดงผลหน้าต่าง GUI และส่งคำสั่งควบคุมจากผู้ใช้

2. ท็อปิกและรูปแบบ Publish-Subscribe (Topics)

Topic คือช่องทางการส่งข้อมูลแบบอะซิงโครนัส ผู้ส่ง (Publisher) ทำการกระจายข้อมูลโดยไม่จำเป็นต้องทราบว่าใครกำลังรอรับ และผู้รับ (Subscriber) จะทำงานอัตโนมัติเมื่อมีข้อมูลใหม่เข้ามา

ชื่อ Topic	Message Type	Publisher	Subscriber	คำอธิบาย
/usb_cam/image_raw	sensor_msgs/Image	usb_cam	detection_node	สตรีมภาพดิบจากกล้อง USB (640x480 @ 30 FPS, YUYV mmap)
/usb_cam/camera_info	sensor_msgs/CameraInfo	usb_cam	detection_node	ค่าพารามิเตอร์การкалиเบรตของเลนส์และเมทริกซ์กล้อง
/detected_objects	std_msgs/String (JSON)	detection_node	Dobot_UI , dobot_controller	รายการบล็อกพิกัดจริง (X, Y, Z), สี, และสถานะการล็อกกริด
/detected_objects_image	sensor_msgs/Image	detection_node	rqt_image_view	สตรีมภาพพร้อมเส้นกรอบตารางข้อมูลพิกัด HUD (เปิดดูผ่าน rqt)
/detected_objects_image/compressed	sensor_msgs/CompressedImage	detection_node	Dobot_UI	สตรีมภาพแบบบีบอัด JPEG ความหน่วงต่ำสำหรับแสดงผลบน GUI
/detected_objects_poses	geometry_msgs/PoseArray	detection_node	RViz / Navigation	พิกัด 3 มิติของบล็อกทั้งหมดในแกนฐานแขนกล (dobot_base)
/grid_local_poses	geometry_msgs/PoseArray	detection_node	โมดูลภายนอก	พิกัด 3 มิติของบล็อกเทียบกับจุดเริ่มต้นของตาราง (grid_origin)
/dobot_status	std_msgs/String (JSON)	dobot_controller	Dobot_UI	สถานะแขนกลพิกัดปัจจุบัน X, Y, Z, R, ลมดูด และเฟสภารกิจ
/dobot_ui_cmd	std_msgs/String (JSON)	Dobot_UI	dobot_controller	คำสั่งควบคุม เช่น เริ่มภารกิจ, จี้ออก, โหมดนิ่ง, หยุดฉุกเฉิน
/set_grid_corners	std_msgs/String (JSON)	calibrate_grid , UI	detection_node	อัปเดตพิกัดมุม 4 มุมของตารางแบบเรียลไทม์

3. ระบบพารามิเตอร์และการเปิดระบบด้วย Launch File

ใน ROS 1 ค่าพารามิเตอร์จะถูกบันทึกไว้บน ROS Parameter Server โดยโหนดสามารถดึงค่าที่เป็นส่วนตัวได้ผ่านเครื่องหมาย ~ (เช่น `rospy.get_param('~image_topic')`) ในไฟล์ Launch (.launch) จะโหลดไฟล์ YAML ผ่านคำสั่ง `<rosparam`

`command="load" file="..." />` ซึ่งไฟล์ YAML ใน ROS 1 จะต้องเป็นโครงสร้างแบบ Flat Dictionary (**ไม่มี** `/**:`
`ros__parameters:` **เหมือนใน** ROS 2) เพื่อให้ค่าถูกโหลดเข้าสู่ Namespace ของโหนดได้อย่างถูกต้อง

บทที่ 3: ระบบประมวลผลภาพด้วย OpenCV และการส่งภาพใน ROS 1

3.1 การจับภาพผ่าน V4L2 และโหนด usb_cam

ในระบบปฏิบัติการ Linux การเชื่อมต่อกล้อง USB จะทำงานผ่านไดรเวอร์ Video4Linux2 (V4L2) โดยโหนด `usb_cam_node` ถูกตั้งค่าให้ดึงภาพจาก `/dev/video0` ด้วยโหมด Memory-Mapped I/O (`io_method: "mmap"`) และฟอร์แมตพิกเซล YUYV ที่ความละเอียด 640 x 480 พิกเซล 30 FPS ซึ่งช่วยลดการใช้ CPU และส่งภาพได้ต่อเนื่องโดยไม่มีอาการกระตุก

3.2 ทำไมต้องใช้พื้นที่สี HSV แทน RGB/BGR?

ภาพจากกล้องทั่วไปจะเป็นระบบ BGR ซึ่งเมื่อแสงสว่างในห้องเปลี่ยนหรือมีเงาตกกระทบ ค่าในทั้ง 3 แชนแนลจะเปลี่ยนไปพร้อมกันทั้งหมด ทำให้การตัดค่าสีไม่เสถียร

ในทางกลับกัน ระบบสี HSV (Hue, Saturation, Value) จะแยกองค์ประกอบของสีออกจากความสว่าง:

- H (Hue [0..180]): บ่งบอกเนื้อสีแท้จริง เช่น สีเขียวจะอยู่ในช่วง 36..85 เสมอไม่ว่าห้องจะสว่างหรือมืด ส่วนสีแดงจะอยู่ที่ช่วง 0..10 และ 170..180
- S (Saturation [0..255]): ความสดของสี ช่วยแยกบล็อกพลาสติกสีสดออกจากผิวกระดาษสีขาว
- V (Value [0..255]): ความสว่าง ช่วยกรองเงาดำมืดออกจากการคำนวณ

3.3 การกรอง Contour และรูปทรงเรขาคณิต

หลังจากสร้าง Binary Mask ด้วย `cv2.inRange()` ระบบจะใช้ Morphological Opening เพื่อลบสัญญาณรบกวน จากนั้นจะตรวจสอบคุณสมบัติทางเรขาคณิต:

- **ขนาดพื้นที่ (Area):** ต้องอยู่ระหว่าง 250 ถึง 30,000 พิกเซล
- **อัตราส่วนด้าน (Aspect Ratio):** ไม่เกิน 2.2 เท่า
- **ความหนาแน่นเนื้องูรูปทรง (Solidity):** พื้นที่รูปทรงเทียบกับ Convex Hull ต้องมากกว่าหรือเท่ากับ 0.78
- **การกรองวงกลมสีเขียวบนกระดาษ (Circularity):** วงแหวนสีเขียวของช่อง Feeder บนกระดาษมีค่า Circularity สูง ระบบจึงตัดทิ้งหากค่าเกิน 0.84 ทำให้ตรวจจับเฉพาะบล็อกลูกเต๋า 3 มิติเท่านั้น

3.4 การตรวจจับและล็อกภาดกริดอัตโนมัติ (Pallet Locking)

ขอบภาดสีเทาจะถูกแยกออกมาด้วยค่าความอิ่มตัวสีต่ำ ($S \leq 50$) และความสว่างปานกลาง (V อยู่ระหว่าง 105 ถึง 195) จากนั้นจะหาพื้นที่ 4 มุมและเรียงลำดับตามเข็มนาฬิกา: P0 (ล่างขวา), P1 (ล่างซ้าย), P2 (บนซ้าย), P3 (บนขวา) มีการใช้ Exponential Moving Average ($\alpha = 0.25$) เพื่อเฉลี่ยพิกัดมุมให้เรียบเนียน และเมื่อมุมอยู่นิ่งครบ 10 เฟรม ระบบจะทำการล็อกพิกัดอัตโนมัติเพื่อป้องกันภาพสั่นไหว

3.5 การแปลงพิกัดภาพเป็นมิลลิเมตรด้วย Homography

เนื่องจากกล้องติดตั้งมุมเอียง ระยะทางในภาพจึงมีความบิดเบี้ยวทางทัศนมิติ ระบบจึงใช้เมทริกซ์ Homography ขนาด 3x3 (`cv2.getPerspectiveTransform`) เพื่อแปลงจุดพิกเซล (u, v) ไปเป็นพิกัดระยะทางที่แท้จริง (gx, gy) ในหน่วยมิลลิเมตร และแปลงต่อเข้าสู่แกนอ้างอิงของฐานแขนกล Dobot (rx, ry, rz) ได้ทันที

3.6 การแปลงภาพเข้าสู่ ROS 1 โดยตรง (ป้องกันปัญหา cv_bridge)

เพื่อแก้ปัญหาความเข้ากันไม่ได้ระหว่าง OpenCV เวอร์ชันใหม่กับไลบรารีของ `cv_bridge` บน Ubuntu 20.04 (ซึ่งมักเกิดข้อผิดพลาด `KeyError: 16`) โค้ด `detection_node.py` ได้ใช้ฟังก์ชันแปลงไบต์อาร์เรย์ NumPy เป็น `sensor_msgs/Image` โดยตรง ทำให้การส่งภาพความเร็วสูงทั้งภาพดิบและภาพบีบอัดทำงานได้อย่างราบรื่น 100%

บทที่ 4: การควบคุมแขนกล Dobot Magician ด้วย pydobot2

4.1 โพรโทคอลการสื่อสาร Serial UART

แขนกล Dobot Magician สื่อสารผ่านพอร์ต USB-to-UART ด้วยความเร็ว 115200 bps ส่งข้อมูลแบบแพ็กเก็ตไบนารี: Header (0xAA 0xAA) | Length | Command ID | Payload | Checksum โดยมีโมดูล `dobot_driver.py` คอยจัดการคิวคำสั่งและค้นหาพอร์ตเชื่อมต่ออัตโนมัติ

4.2 ขอบเขตพื้นที่ทำงานเชิงกล (Cartesian Limits)

ขอบเขตพิกัดที่ปลอดภัยของ Dobot Magician

- X (มม.): ระยะยัดแขนหน้า-หลัง อยู่ระหว่าง 100.0 มม. ถึง 330.0 มม.
- Y (มม.): ระยะหมุนซ้าย-ขวา อยู่ระหว่าง -250.0 มม. ถึง +250.0 มม.
- Z (มม.): ความสูงแนวตั้ง อยู่ระหว่าง -60.0 มม. ถึง +160.0 มม.
- R (องศา): มุมหมุนหัวจับ อยู่ระหว่าง -180.0° ถึง +180.0°

4.3 รูปแบบการเคลื่อนที่ PTP: MOVJ เทียบกับ MOVL

- **MOVJ_XYZ** (Joint Movement): ข้อต่อทุกแกนหมุนพร้อมกัน แขนกลจะเคลื่อนที่เป็นเส้นโค้งด้วยความเร็วสูง เหมาะสำหรับการเคลื่อนที่ข้ามพื้นที่โล่ง
- **MOVL_XYZ** (Linear Movement): ปลายแขนกลเคลื่อนที่เป็นเส้นตรงแนวตั้ง 90° เหมาะสำหรับการเคลื่อนที่ลงไปดูดจับบล็อกลอยและยกบล็อกขึ้นในแนวตรงเพื่อไม่ให้ชนบล็อกลอยข้างเคียง

4.4 การควบคุมหัวดูดสุญญากาศ (End-Effector)

หัวจับของหุ่นยนต์ใช้ระบบดูดสุญญากาศ (Suction Cup) พร้อมการหน่วงเวลาแรงดัน (หน่วงเวลา 0.35 วินาทีหลังเปิดปั๊ม และ 0.25 วินาทีหลังปิดปั๊ม) เพื่อให้แน่ใจว่าบล็อกถูกยึดแน่นหนา ก่อนยกแขนขึ้น

บทที่ 5: สถาปัตยกรรมส่วนติดต่อผู้ใช้ด้วย PyQt5

5.1 การแยกการทำงานด้วย QThread เพื่อป้องกัน UI ค้าง

หน้าจอ PyQt5 ทำงานบน Event Loop หากนำอุปของ ROS (`rospy.spin()`) มารันบนแรมดหลักจะทำให้หน้าต่างค้างและไม่ตอบสนองต่อการคลิกเมาส์

วิธีแก้ไข: แยกการสื่อสาร ROS 1 ไปไว้บน Background Worker Thread ผ่านคลาส `RosBridge(QThread)` :

1. **Main UI Thread:** ทำหน้าที่วาดกราฟิกและรับการกดปุ่มจากผู้ใช้
2. **Worker Thread (`RosBridge`):** ทำหน้าที่รับส่ง Topic จาก ROS 1 ในเบื้องหลัง และส่งข้อมูลมายังหน้าจอผ่านสัญญาณ `pyqtSignal` ที่มีความปลอดภัยต่อการทำงานแบบหลายแรมด (Thread-Safe)

5.2 การพัฒนาระบบด้วย PyQt5 แท้

ใน ROS 1 Noetic บน Python 3.8 การใช้ PyQt6 จะทำให้เกิดปัญหาเรื่องไลบรารีและไวยากรณ์ ระบบจึงถูกปรับเปลี่ยนมาใช้ PyQt5 ล้วน พร้อมคำคอมพิลและ Enum มาตรฐาน ทำให้โปรแกรมทำงานได้อย่างเสถียร

5.3 การรองรับตัวอักษร Unicode และอีโมจิบน Linux

ระบบติดตั้งแพ็คเกจฟอนต์ `fonts-noto-color-emoji` และ `fonts-symbola` พร้อมระบุ `'Noto Color Emoji'` ในสไตล์ชีต CSS ของโปรแกรม ทำให้ไอคอนต่าง ๆ (, , , , , , ) แสดงผลได้อย่างคมชัดสวยงาม ไม่เป็นกล่องสี่เหลี่ยมว่างเปล่า

บทที่ 6: ตรรกะภารกิจ 3 เฟส และการหลบหลีกสิ่งกีดขวาง

6.1 ลำดับการทำงานอัตโนมัติ 3 เฟส (3-Phase Mission)

สถาปัตยกรรมภารกิจ 3 เฟส

1. **เฟส 1 (Clear Obstacles):** คัดแยกบล็อกที่ไม่ใช่เป้าหมายออกจากตาราง แล้วนำไปจัดเก็บไว้ที่ช่อง Feeder 1..4 ด้านซ้าย
2. **เฟส 2 (Stack Goal):** ดูดจับบล็อกเป้าหมายตามลำดับ (#1 ถึง #4) นำมาวางซ้อนกันที่จุด Center Goal [1, 1] สูงสุด 4 ชั้น (ความสูง $Z = 30.0, 55.0, 80.0, 105.0$ มม.)
3. **เฟส 3 (Restore Obstacles):** เมื่อซ้อนบล็อกเป้าหมายเสร็จสิ้น จะนำบล็อกสิ่งกีดขวางจากช่อง Feeder กลับมาวางคืนตำแหน่งเดิมในตาราง

6.2 ระบบคำนวณเส้นทางหลบหลีกการชนเสาบล็อก (Goal Collision Avoidance)

เมื่อบล็อกเป้าหมายถูกซ้อนครบ 4 ชั้น เสาบล็อกจะมีความสูงถึง 130 มม. หากแขนกลเคลื่อนที่เป็นเส้นตรงระหว่างช่อง Feeder ($Y = +79$ มม.) ไปยังตารางฝั่งขวา ($Y = -35$ มม.) ปลายแขนกลจะชนเสาเป้าหมายล้มทับที่

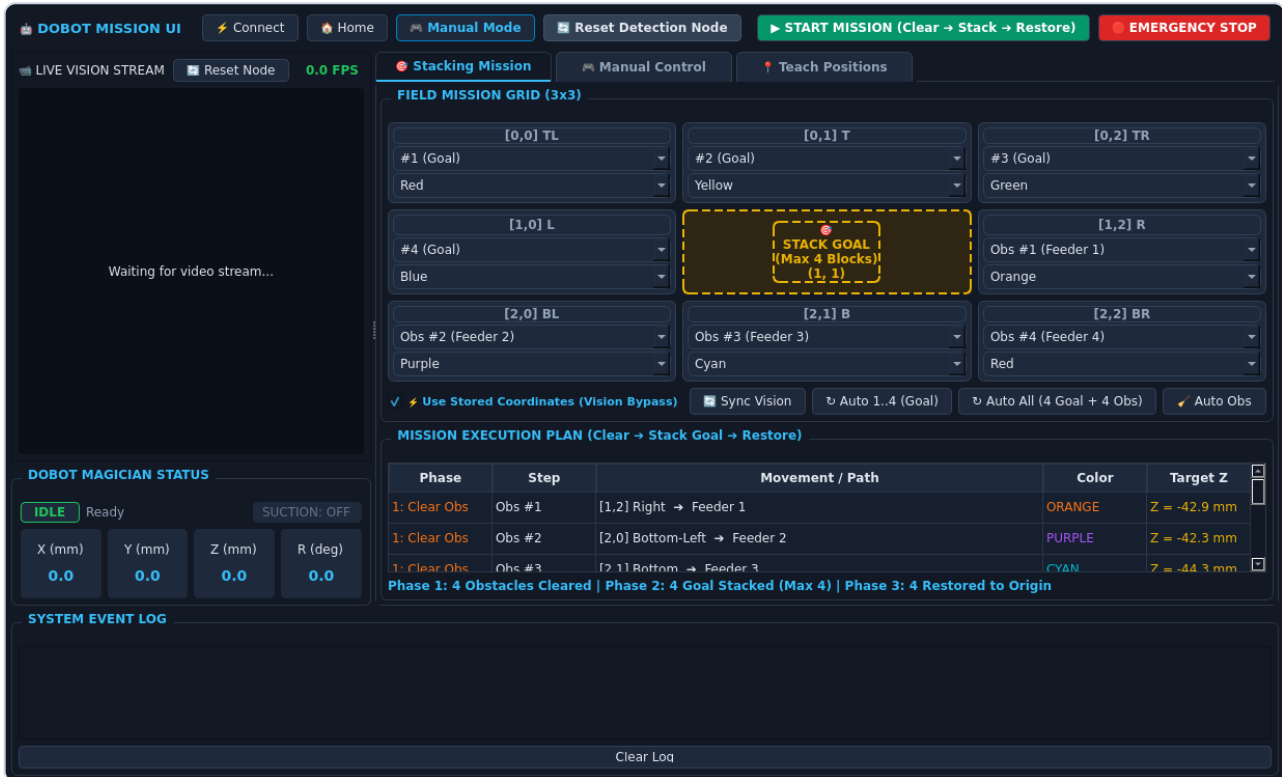
ระบบการสร้างเส้นทางหลบหลีกอัจฉริยะ:

- กำหนดพื้นที่ห้ามเข้าทรงกระบอก (Keepout Zone รัศมี 38.0 มม.) รอบจุด Center Goal (176.0, 0.0)
- ฟังก์ชัน `_point_to_segment_dist()` จะคำนวณตรวจสอบว่าเส้นทางตรงตัดผ่านพื้นที่ห้ามเข้าหรือไม่
- หากพบการตัดผ่าน ฟังก์ชัน `_plan_safe_transit()` จะสร้างจุดแวะเลี้ยวหลบขอบตาราง:
 - **ทางเลี้ยวทิศเหนือ (North Bypass):** เลี้ยวอ้อมด้านบนที่แกน $X = 222.0$ มม.
 - **ทางเลี้ยวทิศใต้ (South Bypass):** เลี้ยวอ้อมด้านล่างที่แกน $X = 132.0$ มม.
- **การยกระดับความสูงปลอดภัย:** หากมีบล็อกอยู่ที่จุด Goal ระบบจะยกระดับความสูงขณะเคลื่อนย้ายขึ้นเป็น $Z = 142.0$ มม. โดยอัตโนมัติ เพื่อลอยข้ามเสาบล็อกสูง 130 มม. ได้อย่างปลอดภัย

บทที่ 7: คู่มือการใช้งานหน้าต่างโปรแกรม (UI Walkthrough)

7.1 แท็บภารกิจซ้อนบล็อก (Stacking Mission Tab)

หน้าจอหลักสำหรับการตั้งค่าภารกิจ เลือกลำดับบล็อกเป้าหมาย และเริ่มการทำงานแบบอัตโนมัติ:

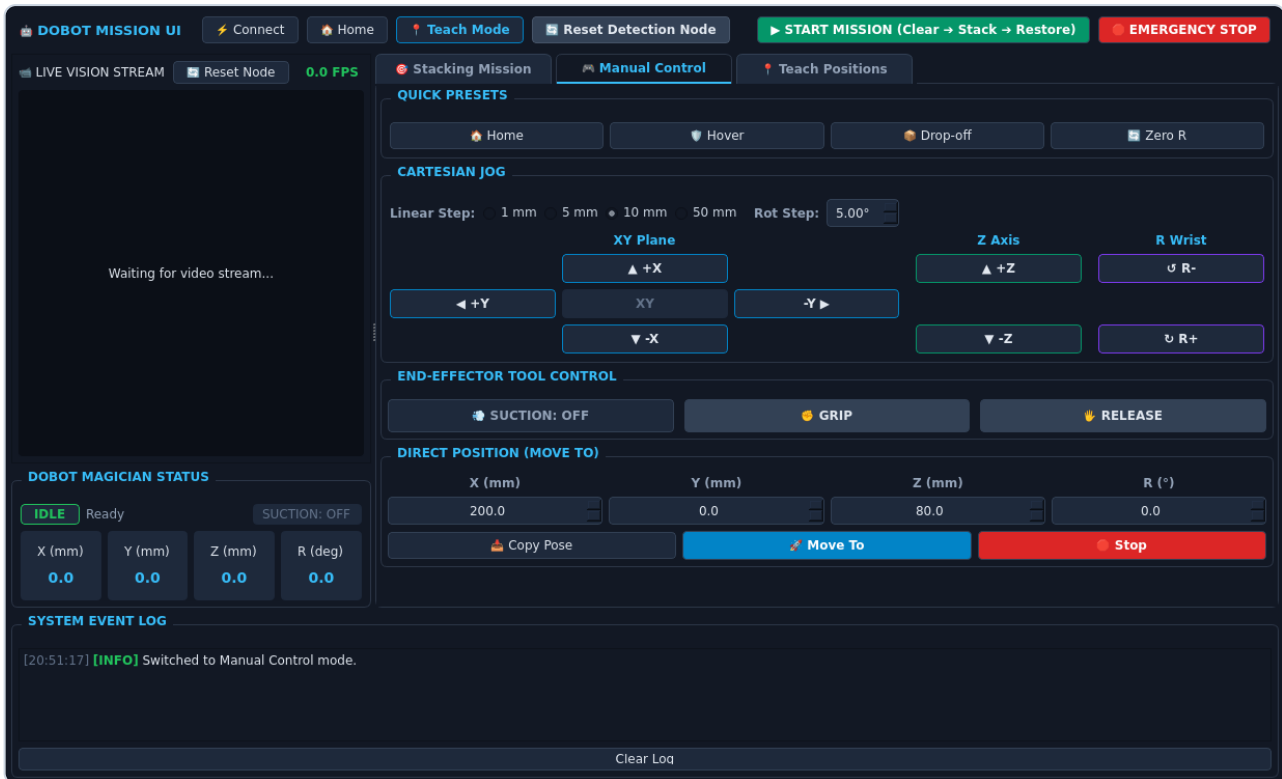


รูปที่ 7.1: หน้าต่างจัดการภารกิจ พร้อมภาพสตรีมกล้องสด ตารางตำแหน่ง และตารางแผนการทำงาน

- **ปุ่มกำหนดอัตโนมัติ:** **Auto 1..4 (Goal)** กำหนดบล็อกเป้าหมายอัตโนมัติ; **Auto All** กำหนดบล็อกทั้ง 8 ก้อน; **Auto Obs** สั่งเคลียร์สิ่งกีดขวาง
- **ตารางลำดับงาน (Execution Plan):** แสดงรายการขั้นตอนทั้ง 3 เฟส พร้อมแถบสีแจ้งสถานะการทำงานจริง

7.2 แท็บควบคุมด้วยมือ (Manual Control Tab)

ใช้สำหรับทดสอบการเคลื่อนที่รายแกน ตั้งพิกัดล่วงหน้า และควบคุมหัวดูดจับ:

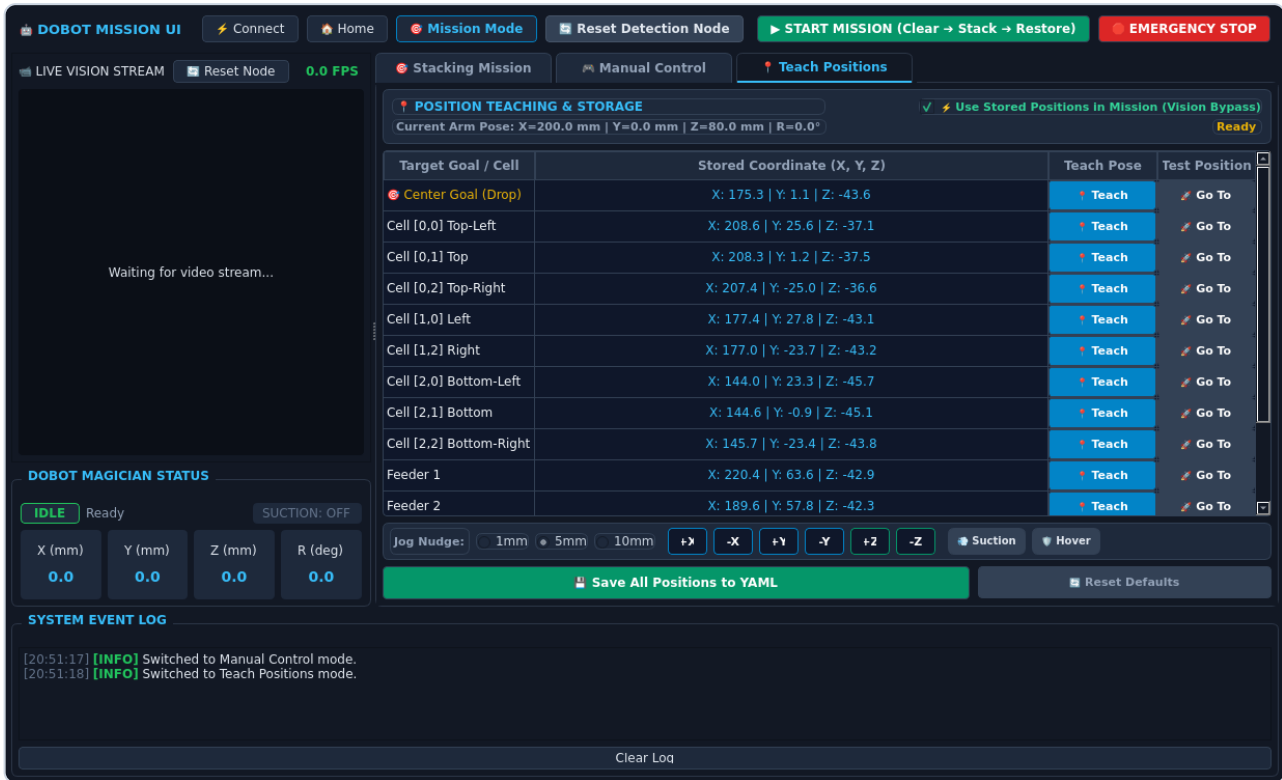


รูปที่ 7.2: แผงควบคุมแบบ Jogging พิกัด X, Y, Z, R และปุ่มเปิด-ปิดระบบสุญญากาศ

- **ปุ่มพิกัดลัด (Quick Presets):** สั่งแขนกลไปยังตำแหน่ง Home , Hover , Drop-off , และ Zero R ได้ในคลิกเดียว
- **ปุ่มจ็อกกิ้ง (Cartesian Jog):** ขยับแขนกลทีละแกน พร้อมเลือกระยะก้าวได้ (1, 5, 10, 50 มม.)
- **ควบคุมหัวดูด:** ปุ่มเปิด/ปิดลมดูด (SUCTION: ON/OFF) และสั่งเปิด/ปิดมือจับกล่อง

7.3 แท็บสอนฝึกและระบบสำรอง (Teach Positions Tab)

ใช้สำหรับบันทึกค่าพิกัดแต่ละช่องด้วยมือ เพื่อใช้เป็นระบบสำรองเมื่อกล่องถูกบัง:



รูปที่ 7.3: ตารางบันทึกพิกัดตำแหน่ง และปุ่มบันทึกค่าลงไฟล์คอนฟิก YAML

- ขยับปลายแขนกลไปแตะที่ช่องตารางหรือช่อง Feeder แล้วกดปุ่ม **Teach** เพื่อบันทึกพิกัดจริง
- กดปุ่ม **Save All Positions to YAML** เพื่อบันทึกค่าลงไฟล์ `dobot_ui.yaml` อย่างถาวร
- **ระบบข้ามการตรวจจับกล่อง (Vision Bypass):** หากติ๊กเลือกช่องนี้ แขนกลจะทำงานตามพิกัดที่สอนไว้ทันที เหมาะสำหรับใช้งานในกรณีที่แสงในห้องเปลี่ยนแปลง กล่องหลุด หรือระบบภาพขัดข้อง

บทที่ 8: คำสั่งเริ่มต้นใช้งานและการแก้ไขปัญหา

8.1 คำสั่งคอมไพล์และเปิดระบบ (ROS 1 Quick Start)

```
# 1. โหลด Environment ของ ROS 1 Noetic และคอมไพล์ Catkin Workspace
cd /root/Dobot-Magician-Autonomous-Stacking-ROS1
source /opt/ros/noetic/setup.bash
catkin_make
source devel/setup.bash

# 2. เปิดระบบทั้งหมดพร้อมกันด้วย Launch File (กล่อง + ตรวจจจับภาพ + ตัวควบคุมแขนกล + UI)
roslaunch dobot_v2 dobot_system.launch

# 3. หรือเปิดเฉพาะระบบประมวลผลภาพกล่อง
roslaunch dobot_v2 dobot_vision.launch

# 4. ตรวจสอบสตรีมภาพและข้อมูลตรวจจับผ่าน rqt_image_view
rqt_image_view /detected_objects_image

# 5. หรือเปิดเฉพาะหน้าต่าง GUI ในโหมดจำลอง (Mock Simulation Mode โดยไม่ต้องต่อฮาร์ดแวร์จริง)
roslaunch Dobot_UI dobot_ui.launch mock:=true
```

8.2 ปัญหาที่พบบ่อยและวิธีแก้ไข (Troubleshooting)

💡 ปัญหาการเชื่อมต่อพอร์ต USB Serial บน Linux

หากต่อแล้วขึ้นข้อผิดพลาด Permission denied: '/dev/ttyUSB0':

```
sudo usermod -a -G dialout $USER
sudo chmod 666 /dev/ttyUSB0
# จากนั้นให้ออกจากระบบ (Logout) แล้วเข้าสู่ระบบใหม่เพื่อให้สิทธิ์มีผล
```

💡 ปัญหากล้องค้างหรือขึ้น Device Busy

หากกล้องเปิดไม่ติดหรือภาพค้าง ให้กดปุ่ม  Reset Detection Node บนแถบด้านบนของ GUI เพื่อสั่งรีเซ็ตไหนดตรวจสอบภาพใหม่

✅ คำสั่งตรวจสอบการส่งข้อมูลใน ROS 1

ตรวจสอบอัตราการส่งข้อมูลของ Topic ต่าง ๆ ได้ด้วยคำสั่งมาตรฐานของ ROS 1:

```
rostopic hz /detected_objects_image           # ตรวจสอบภาพสด (ควรมีที่ ~20-30 Hz)
rostopic hz /detected_objects_image/compressed# ตรวจสอบภาพบีบอัดสำหรับ UI (~20-30 Hz)
rostopic echo /detected_objects                # ดูข้อมูลพิกัดบล็อกแบบ JSON เรียลไทม์
```

ผู้จัดทำ: Thanadech (Google Deepmind Pair-Programming Assistant - Antigravity) • กันยายน 2026